Software Requirements Specification

for

PathMind: Comparative Pathfinding with AI Insights

Version 0.1 approved

Prepared by

Yahya Musmar

29/9/2025

Table of Contents

Revision History

Name	Date	Reason For Changes	Version
PathMind: Comparative Pathfinding with AI Insights	29/9/2025	Initial requirements documented	Version **0.1**
PathMind: Comparative Pathfinding with AI Insights	-	Final requirements documented	Version 1

1. Introduction

1.1 Purpose

The purpose of this project is to design and implement an interactive Unity-based application that compares four pathfinding algorithms—Depth-First Search (DFS), Breadth-First Search (BFS), Dijkstra's Algorithm, and A*—on various map scenarios. The project integrates a Small Language Model (SLM) to automatically analyze the performance results of each algorithm and provide a brief explanation of which algorithm performed best, the order of performance ranking, and the reasoning behind these results.

All generated results and explanations will be stored in a Microsoft Azure SQL Database for record-keeping, further analysis, and scalability.

This system aims to:

- Help students and researchers understand the differences between common pathfinding algorithms.
- Demonstrate how AI models (SLM) can enhance technical results with natural language explanations.
- Showcase how cloud services (Azure) can be used for storing and managing experiment data.

1.2 Product Scope

The proposed system is an educational and experimental platform developed in **Unity** that demonstrates and compares the performance of four popular pathfinding algorithms: **Depth-First Search (DFS), Breadth-First Search (BFS), Dijkstra's Algorithm, and A***.

The application allows users to:

- Generate or load different grid-based maps with obstacles.
- Run all four algorithms on the same map scenario.
- Observe the calculated paths and execution characteristics (e.g., path length, number of steps, cost).
- Send the performance results to a Small Language Model (SLM), which produces a natural language explanation and ranking of the algorithms.
- Store the results, explanations, and map details into a **Microsoft Azure SQL Database** for future reference and scalability.

The system will serve as a learning and analysis tool for students, educators, and researchers in computer science and artificial intelligence. By combining algorithm visualization, AI-powered explanations, and cloud-based storage, the project demonstrates both theoretical and practical aspects of modern software systems.

1.3 Definitions, acronyms, and abbreviations

- **DFS (Depth-First Search):** A graph traversal algorithm that explores as far as possible along one path before backtracking.
- **BFS (Breadth-First Search):** A graph traversal algorithm that explores all neighbors at the current depth before moving to the next level.
- **Dijkstra's Algorithm:** A shortest-path algorithm that finds the least-cost path from a source to all other nodes in a weighted graph.
- **A* (A-Star Algorithm):** An informed search algorithm that uses cost and heuristic estimates to find the optimal path efficiently.
- **SLM (Small Language Model):** A compact AI model capable of generating natural language explanations or performing limited NLP tasks locally.
- **LLM (Large Language Model):** A more complex AI model trained on large datasets, typically requiring cloud or API access.
- **Azure SQL Database:** A managed cloud database service provided by Microsoft Azure for storing and querying relational data.
- **Unity Engine:** A cross-platform game engine used to develop the application and visualize pathfinding algorithms.
- **API (Application Programming Interface):** A set of functions or protocols that allow software components to communicate with each other.
- **vCore (Virtual Core):** A unit of computing power used in Azure services to measure resource allocation.

1.4 References

- IEEE. *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.* IEEE Computer Society, 1998.
- Unity Technologies. *Unity Documentation.* Available: https://unity.com
- Microsoft Azure. *Azure SQL Database Documentation.* Available: https://azure.microsoft.com
- Hugging Face. *Transformers Documentation.* Available: https://huggingface.co

1.5 Overview

This project is a Unity-based application that compares four pathfinding algorithms—DFS, BFS, Dijkstra, and A*—on various grid maps. A Small Language Model (SLM) analyzes the results, providing a ranking of the algorithms and brief explanations for their performance. All data, including map details, algorithm results, and AI-generated explanations, is stored in an Azure SQL Database.

The system combines algorithm visualization, AI-powered insights, and cloud storage to create an educational and research-oriented tool for understanding pathfinding techniques.

2. Overall Description

2.1 Product Perspective

The system is a **standalone educational and experimental tool** developed in Unity, designed to compare and analyze four pathfinding algorithms on grid-based maps. It integrates a **Small Language Model (SLM)** for automatic explanation of algorithm performance and uses **Azure SQL Database** for storing map data, results, and AI-generated insights. The product functions as an independent application but can interface with external components such as the SLM (local or API-based) and the cloud database. Its modular design allows future expansion, such as adding more algorithms, larger maps, or cloud-based LLM integration, without requiring major architectural changes.

2.2 Product Functions

The system provides the following main functions:

1. **Map Generation and Selection**
    - Users can generate new grid maps or select pre-defined maps with obstacles, start, and goal positions.
2. **Pathfinding Algorithm Execution**
    - The system runs **DFS, BFS, Dijkstra, and A*** on the selected map.
    - Each algorithm calculates the path, path length, execution time, and number of nodes explored.
3. **SLM Analysis**
    - The Small Language Model receives the results and map information.
    - It generates a brief natural language explanation, ranking the algorithms and providing reasoning for performance differences.
4. **Data Storage**
    - All map data, algorithm results, and SLM explanations are stored in **Azure SQL Database** for future reference.
5. **Visualization and User Interaction**
    - Unity displays the paths found by each algorithm.
    - Users can view the AI-generated explanation alongside the visual results.
6. **Future Expandability**
    - The system is designed to allow adding new algorithms, larger maps, or cloud-based AI models with minimal changes.

2.3 User Classes and Characteristics

The system is designed for the following user classes:

1. **Students**
    - Users who are learning about pathfinding algorithms, game development, or AI.
    - Characteristics: Basic familiarity with computers and interest in algorithms.
2. **Educators/Researchers**
    - Users who want to analyze algorithm performance, compare results, or demonstrate AI-assisted analysis.
    - Characteristics: Knowledge of algorithms and data analysis; may require access to stored results for evaluation.
3. **General Enthusiasts**
    - Users interested in experimenting with pathfinding algorithms and AI explanations in a game environment.
    - Characteristics: Casual users with basic computer skills; primarily use the visualization and explanation features.

All users interact with the system via a **graphical interface in Unity**, which provides intuitive controls to select maps, run algorithms, and view results and AI-generated explanations.

2.4 Use Cases

Use Case 1: Run Pathfinding Algorithms on a Map

- **Actor:** Student / Educator
- **Description:** User selects a map or generates a new one. The system runs DFS, BFS, Dijkstra, and A* on the map and calculates path length, nodes explored, and execution time.
- **Outcome:** Paths for each algorithm are displayed, and raw performance results are recorded.

Use Case 2: Generate AI Explanation

- **Actor:** Student / Educator
- **Description:** The system sends the algorithm results and map information to the SLM. The SLM generates a natural language explanation, ranking algorithms by performance and providing reasoning.
- **Outcome:** Explanation is displayed in Unity and stored in Azure SQL Database.

Use Case 3: Store Results in Database

- **Actor:** System (automatic)
- **Description:** All map details, algorithm results, and AI-generated explanations are saved in Azure SQL Database for later retrieval or analysis.

- **Outcome:** Persistent record of experiments for review or comparison.

Use Case 4: Visualize Results

- **Actor:** Student / Educator / Enthusiast
- **Description:** Users can view the algorithm paths on the grid map alongside AI-generated explanations.
- **Outcome:** Users gain insight into algorithm efficiency and behavior in a visual and textual format.

2.5 Operating Environment

The system will operate in the following environments:

1. **Hardware Environment:**
    - Personal computer or laptop with at least:
        - CPU: Intel i5 / AMD Ryzen 5 or higher
        - RAM: 8 GB minimum (16 GB recommended for smoother SLM execution)
        - GPU: Optional but recommended for faster SLM processing
        - Storage: 10 GB free disk space for project files and models
2. **Software Environment:**
    - **Operating System:** Windows 10/11, or modern Linux distributions
    - **Unity Game Engine:** Version 2021 or later
    - **Python Environment:** For running the SLM (Python 3.9+ with `transformers`, `torch`)
    - **Database:** Microsoft Azure SQL Database (free tier)
3. **Network Requirements:**
    - Internet connection is required to access **Azure SQL Database**.
    - Optional: Internet needed if using **Hugging Face API** instead of a local SLM.
4. **User Interface:**
    - Graphical interface built in Unity for map creation, algorithm execution, and visualization of results and AI-generated explanations.

The system is designed to be **cross-platform** where possible and can run on typical student or research lab computers without requiring high-end hardware.

2.6 Design and Implementation Constraints

1. **Hardware Limitations:**

- The system should run on standard student PCs or laptops.

- Large maps or heavy SLM models may require more RAM or a GPU, so the system must handle limited resources gracefully.

2. **SLM Limitations:**

- The Small Language Model must be lightweight enough to run locally or with minimal API calls.
- Model input size and processing time should be kept small to maintain responsiveness.

3. **Unity Engine Constraints:**

- The game engine must support grid-based pathfinding and visualization of multiple algorithms.
- The interface should remain responsive even when multiple algorithms are running simultaneously.

4. **Database Constraints:**

- Azure SQL Database free tier limits (e.g., 100,000 vCore seconds, 32 GB storage) must be considered.
- Data storage and retrieval should be optimized to avoid exceeding free tier limits.

5. **Software Dependencies:**

- Python libraries (`transformers`, `torch`) for the SLM must be compatible with the Unity-to-Python communication method.
- Any API-based SLM usage should comply with free-tier request limits.

6. **Scalability & Future Extensions:**

- The system should be designed modularly to allow future addition of algorithms, larger maps, or cloud-based LLM integration without major architectural changes.

2.7 Assumptions and Dependencies

Assumptions:

1. Users have basic familiarity with computers and Unity interface.
2. The system will be used primarily on PCs or laptops with minimum hardware requirements as specified.
3. Users will have internet access if storing data in Azure SQL Database or using API-based SLM.
4. The maps will be grid-based and of moderate size to ensure algorithms and SLM processing remain efficient.

5. The SLM will produce meaningful explanations given structured input data from the algorithms.

Dependencies:

1. **Unity Game Engine** – required for building and running the application.
2. **Python Environment** – needed for running the SLM locally (or via API if using hosted SLM).
3. **Python Libraries** – `transformers`, `torch` or any other library required for the chosen SLM.
4. **Azure SQL Database** – required for storing experiment data, algorithm results, and SLM explanations.
5. Optional: **Hugging Face API** or other hosted model APIs if not using a fully local SLM.

3. Specific requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system will provide a graphical user interface (GUI) within the Unity game engine. The user interface will allow users to:

- Load or generate a map with obstacles.
- Select and run one of the four pathfinding algorithms (DFS, BFS, Dijkstra, A*).
- View the visualized path taken by the chosen algorithm.
- Compare results from all algorithms in a results panel.
- Click a button to request an explanation from the integrated Small Language Model (SLM).
- View the SLM's explanation and ranking in a text panel within the game.
- Optionally save the results and explanations to the Azure SQL Database.

The interface will be simple, interactive, and optimized for desktop use with mouse and keyboard controls.

3.1.2 Hardware Interfaces

The system will run on standard desktop or laptop computers. Minimum hardware requirements include:

- **Processor:** Dual-core CPU (Intel i5 / AMD equivalent or higher).
- **Memory:** 8 GB RAM or higher.
- **Graphics:** GPU supporting Unity 3D rendering (e.g., DirectX 11 compatible).

- **Storage:** At least 2 GB of free space for Unity project files, database connection drivers, and related assets.
- **Input Devices:** Standard keyboard and mouse.

No special hardware (e.g., sensors or VR equipment) is required for the operation of the system.

3.1.3 Software Interfaces

The system will rely on the following software components:

- **Unity Engine**: The primary development environment for implementing the pathfinding algorithms and game interface.
- **C# Programming Language**: Used within Unity for implementing system logic.
- **Azure SQL Database**: For storing results of pathfinding algorithms, explanations generated by the small/large language model, and user interaction data.
- **Local SLM/LLM Environment**: A lightweight model integrated locally to analyze algorithm results and provide textual explanations.
- **Microsoft Azure Services**: APIs and cloud tools for database access and integration.
- **Operating System**: Windows 10 (or higher) for development and deployment.

3.1.4 Communications Interfaces

The system will require communication between the Unity application, the local SLM/LLM module, and the Azure SQL Database. Data exchange will occur using:

- **SQL Queries**: For sending and retrieving data from the Azure SQL Database.
- **RESTful APIs / SDKs**: For integration with Microsoft Azure cloud services.
- **Local Communication**: Between Unity and the SLM/LLM module through function calls or local APIs.
- **Internet Connectivity**: Required for cloud-based operations, such as saving data to Azure.

3.2 Functional requirements

The system shall provide the following core functionalities:

1. **Map Setup**

- The system shall provide predefined or randomly generated grid-based maps with obstacles, start, and goal positions.
- The user shall be able to select a map to run the algorithms on.

2. **Pathfinding Algorithms Execution**

- The system shall implement **DFS, BFS, Dijkstra's, and A***.
- The system shall execute these algorithms on the selected map.
- The system shall display the paths explored and the final solution.

3. **Algorithm Comparison**

- The system shall compare the algorithms' performance based on path length, time, and efficiency.

4. **Explanation via SLM/LLM**

- The system shall provide results to a local SLM/LLM for analysis.
- The SLM/LLM shall generate a short explanation ranking the algorithms and justifying the ranking.

5. **Cloud Data Storage (Azure SQL Database)**

- The system shall save each run's results, explanations, and map ID into the Azure SQL Database.
- The system shall allow retrieving stored results later.

6. **User Interaction**

- The system shall allow the user to:
    - Select a predefined or random map
    - Run all four algorithms
    - View the results and explanations
    - Save or review results from Azure

3.3 Other Nonfunctional Requirements

3.3.1 Performance Requirements

- The system shall execute each pathfinding algorithm (DFS, BFS, Dijkstra, A*) within **1–2 seconds** on small maps (e.g., 5x5 to 10x10 grids).
- The system shall return explanations from the SLM in **under 2 seconds** after receiving algorithm results.
- The system shall store and retrieve results from the Azure SQL Database with a response time of less than **2 seconds**.
- The system shall handle multiple test runs in sequence without performance degradation.

3.3.2 Safety Requirements

- The system shall include proper error handling to prevent crashes if algorithms fail to find a path or if the database connection is lost.
- The system shall prevent accidental overwriting of previously stored results by assigning a unique identifier to each map run.
- The system shall provide fallback messages from the SLM in case the explanation module fails or produces invalid output.

3.3.3 Security Requirements

- The system shall not store sensitive user information; only experiment data (maps, algorithm results, SLM explanations) will be saved.
- Communication between Unity and Azure SQL Database shall use **encrypted connections** (e.g., TLS/SSL) to prevent interception of data.
- Local SLM files and scripts shall be protected from accidental modification or deletion.

3.3.4 Software Quality Attributes

- **Usability:** The Unity interface shall be intuitive and easy to use, allowing users to select maps, run algorithms, and view results with minimal learning curve.
- **Availability:** The system shall be accessible whenever the user runs the application, and critical components (Unity app, SLM, database) shall function reliably during use.
- **Maintainability:** The system shall have a modular design to allow easy updates, such as adding new algorithms or updating the SLM module.
- **Portability:** The system shall run on standard Windows PCs and, if feasible, on modern Linux systems without major changes.
- **Reliability:** The system shall consistently provide correct algorithm results and accurate SLM explanations under normal operating conditions.
- **Interoperability:** The system shall be able to communicate and exchange data smoothly between Unity, the SLM module, and Azure SQL Database.
- **Scalability:** The system shall allow future expansion to larger maps, more algorithms, or cloud-based LLM integration without significant redesign.
- **Testability:** The system shall be designed to allow easy testing of algorithms, SLM explanations, and database storage/retrieval.

3.4 Logical database requirements

The system will use **Azure SQL Database** to store data for all map runs and SLM explanations. The logical design includes the following tables and relationships:

1. **Maps Table**
    - **MapID** (Primary Key): Unique identifier for each map
    - **MapData**: Representation of the grid map, including obstacles, start, and goal positions
    - **MapName**: Optional descriptive name for the map
2. **Algorithms Table**
    - **AlgorithmID** (Primary Key): Unique identifier for each algorithm (DFS, BFS, Dijkstra, A*)
    - **AlgorithmName**: Name of the algorithm
3. **Results Table**
    - **ResultID** (Primary Key)
    - **MapID** (Foreign Key): References Maps table
    - **AlgorithmID** (Foreign Key): References Algorithms table
    - **PathLength**: Number of steps in the path
    - **NodesExplored**: Number of nodes visited during execution
    - **ExecutionTime**: Time taken to complete the pathfinding
4. **SLM_Explanations Table**
    - **ExplanationID** (Primary Key)
    - **ResultID** (Foreign Key): References Results table
    - **ExplanationText**: The text generated by the SLM
    - **AlgorithmRanking**: Rank of the algorithm relative to others for the same map

Additional Notes:

- Relationships: Each map can have multiple results (one per algorithm), and each result has a single explanation from the SLM.
- Data types: All IDs are integers, ExecutionTime is float, PathLength and NodesExplored are integers, and ExplanationText is text/varchar.
- The database must support **insertion and retrieval** operations efficiently for multiple map runs.

Data Entities and Their Relationships:

The system's database consists of four main entities: **Maps, Algorithms, Results, and SLM_Explanations**. Their relationships are as follows:

1. **Maps**
    - **Attributes:** MapID (PK), MapData, MapName

- **Description:** Represents each unique map used for algorithm testing.
- **Relationships:** One map can have **multiple Results** (one per algorithm run).

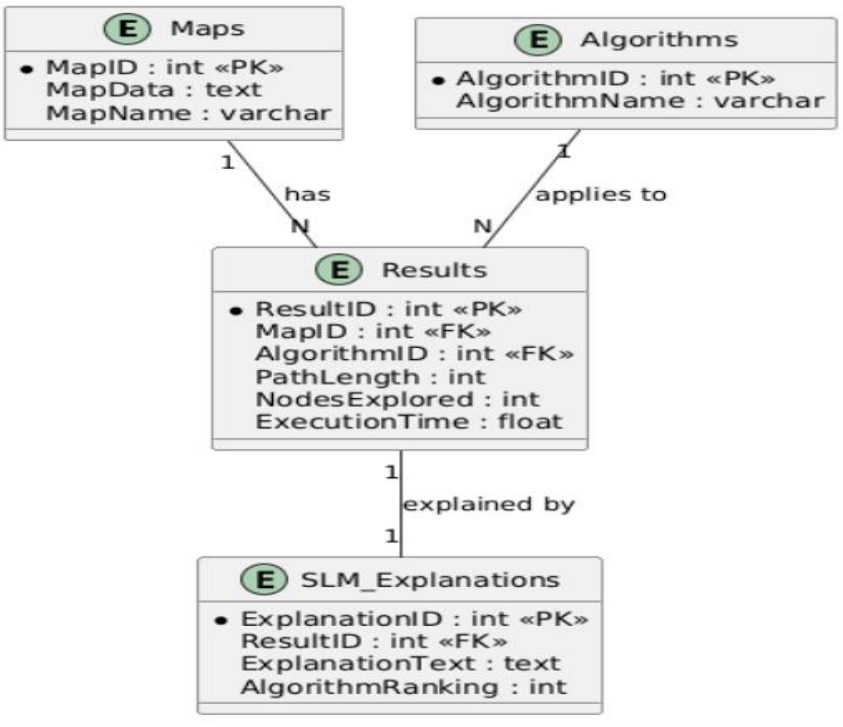
2. **Algorithms**
   - **Attributes:** AlgorithmID (PK), AlgorithmName
   - **Description:** Represents the pathfinding algorithms implemented in the system (DFS, BFS, Dijkstra, A*).
   - **Relationships:** Each algorithm can appear in **multiple Results** for different maps.
3. **Results**
   - **Attributes:** ResultID (PK), MapID (FK), AlgorithmID (FK), PathLength, NodesExplored, ExecutionTime
   - **Description:** Stores the outcome of a specific algorithm run on a specific map.
   - **Relationships:** Each result belongs to **one Map** and **one Algorithm**. Each result has **one associated SLM explanation**.
4. **SLM_Explanations**
   - **Attributes:** ExplanationID (PK), ResultID (FK), ExplanationText, AlgorithmRanking
   - **Description:** Stores the natural language explanation generated by the SLM, including ranking of algorithms.
   - **Relationships:** Each explanation is linked to **one Result**.

Entity Relationship Summary:

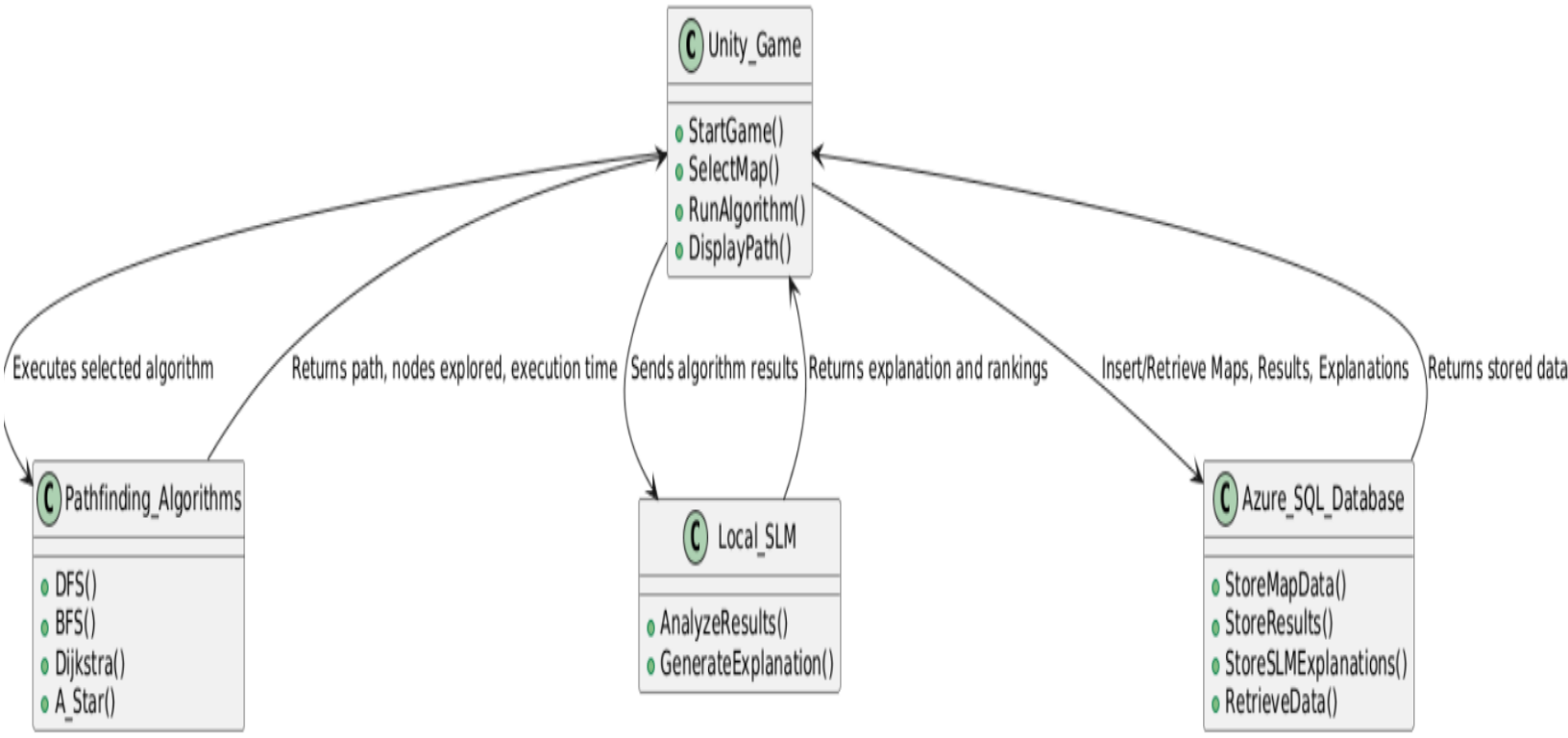
- **Maps 1 — N Results**
- **Algorithms 1 — N Results**
- **Results 1 — 1 SLM_Explanations**

This design ensures that every algorithm run on a map is captured, analyzed by the SLM, and stored for later retrieval.

ERD BE LIKE :-



4. Architectural design

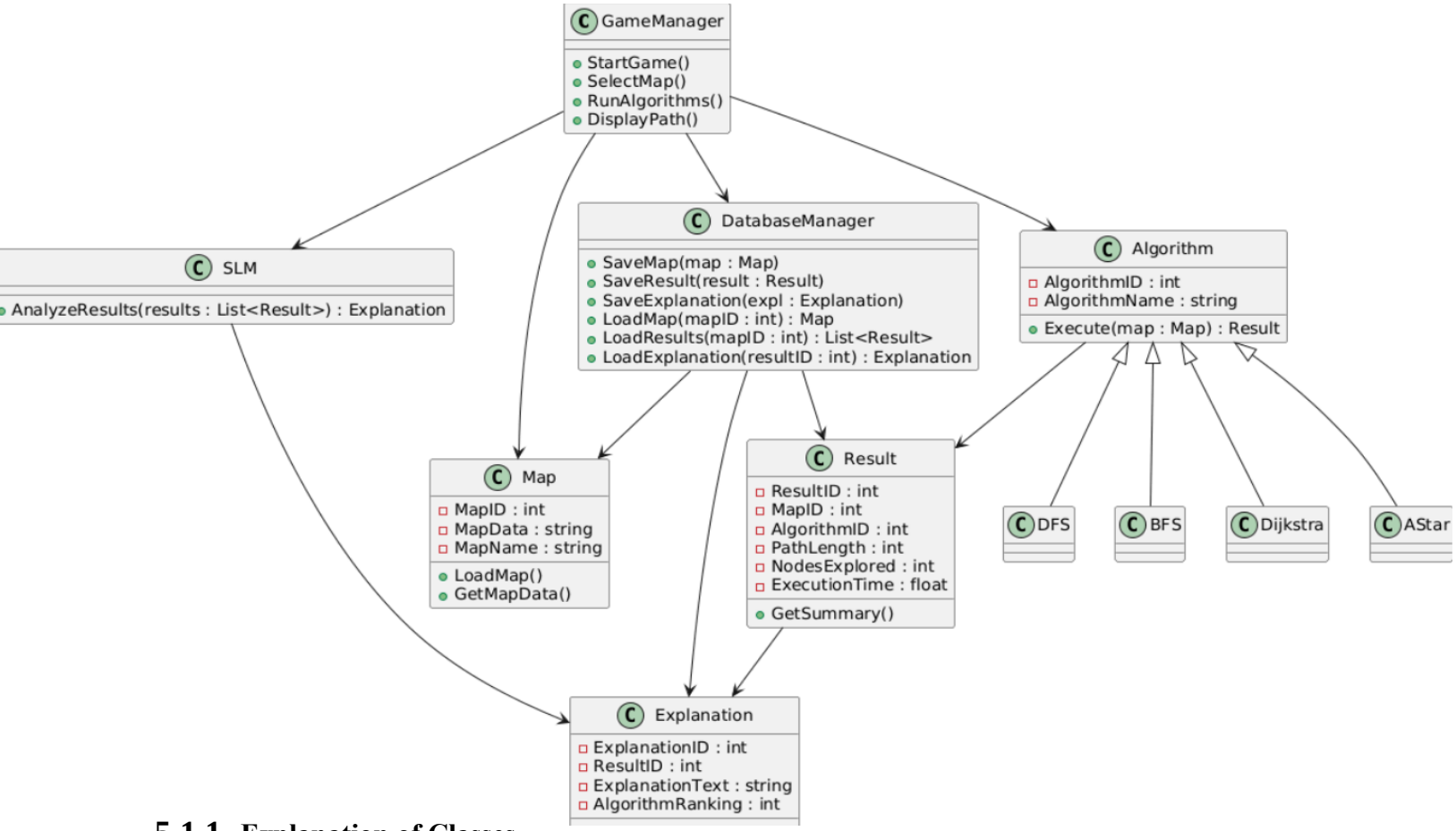


4.1.1 Explanation of Components:

1. **Unity_Game**: The main application that handles user input, map selection, and displays paths.
2. **Pathfinding_Algorithms**: Executes DFS, BFS, Dijkstra, and A* on the selected map.
3. **Local_SLM**: Receives algorithm results and generates **explanations and rankings**.
4. **Azure_SQL_Database**: Stores all maps, results, and explanations for retrieval.

5. System Models

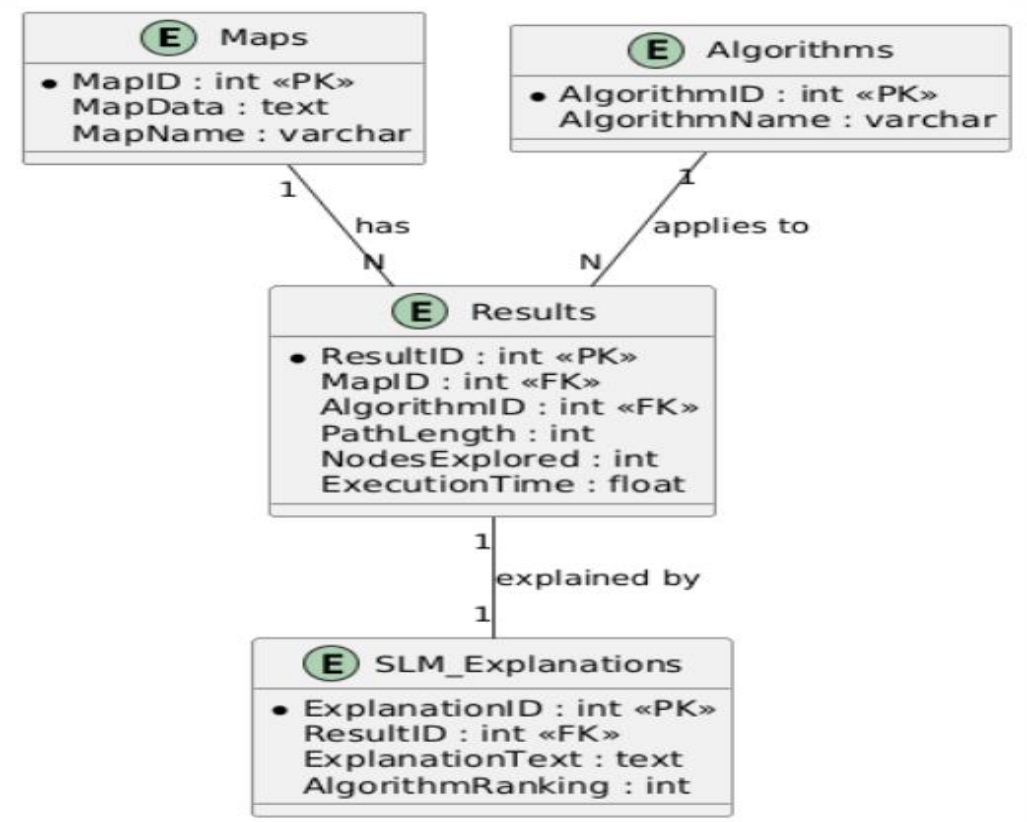
5.1 Class diagrams



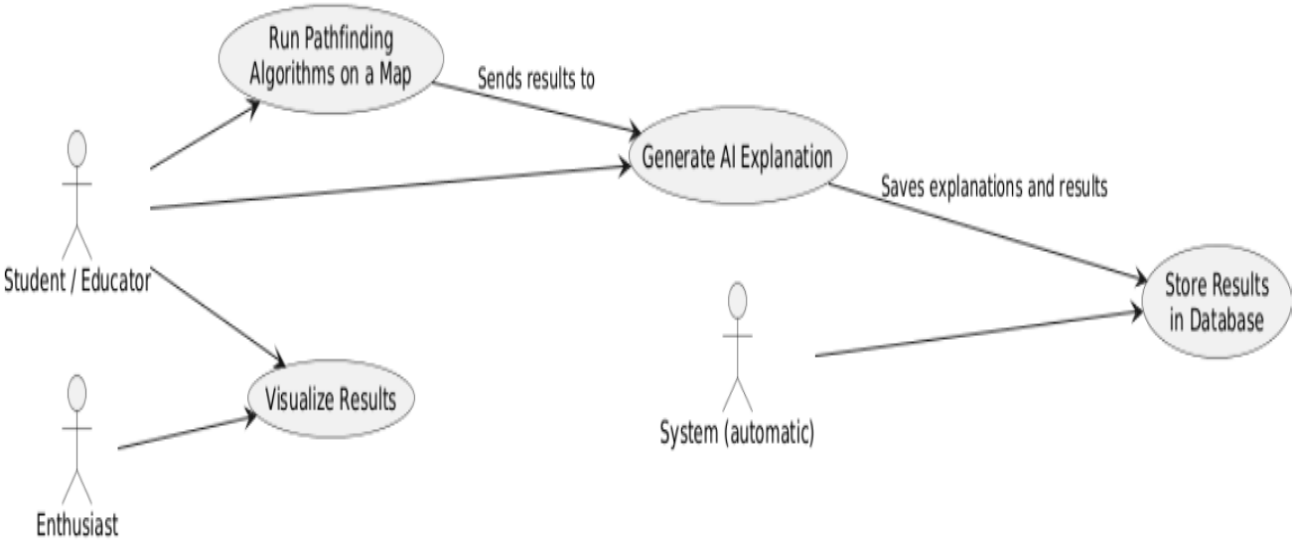
5.1.1 Explanation of Classes

1. **GameManager**
   - Central controller for Unity: handles user input, runs algorithms, displays paths.
2. **Map**
   - Represents the grid map with obstacles, start and goal positions.
3. **Algorithm & Subclasses (DFS, BFS, Dijkstra, A)***
   - Base Algorithm class defines interface; subclasses implement specific pathfinding logic.
4. **Result**
   - Stores algorithm execution outcome: path length, nodes explored, execution time.
5. **SLM & Explanation**
   - Local SLM class analyzes results and produces natural language explanations with rankings.
6. **DatabaseManager**
   - Handles all database operations: storing and retrieving Maps, Results, and Explanations.

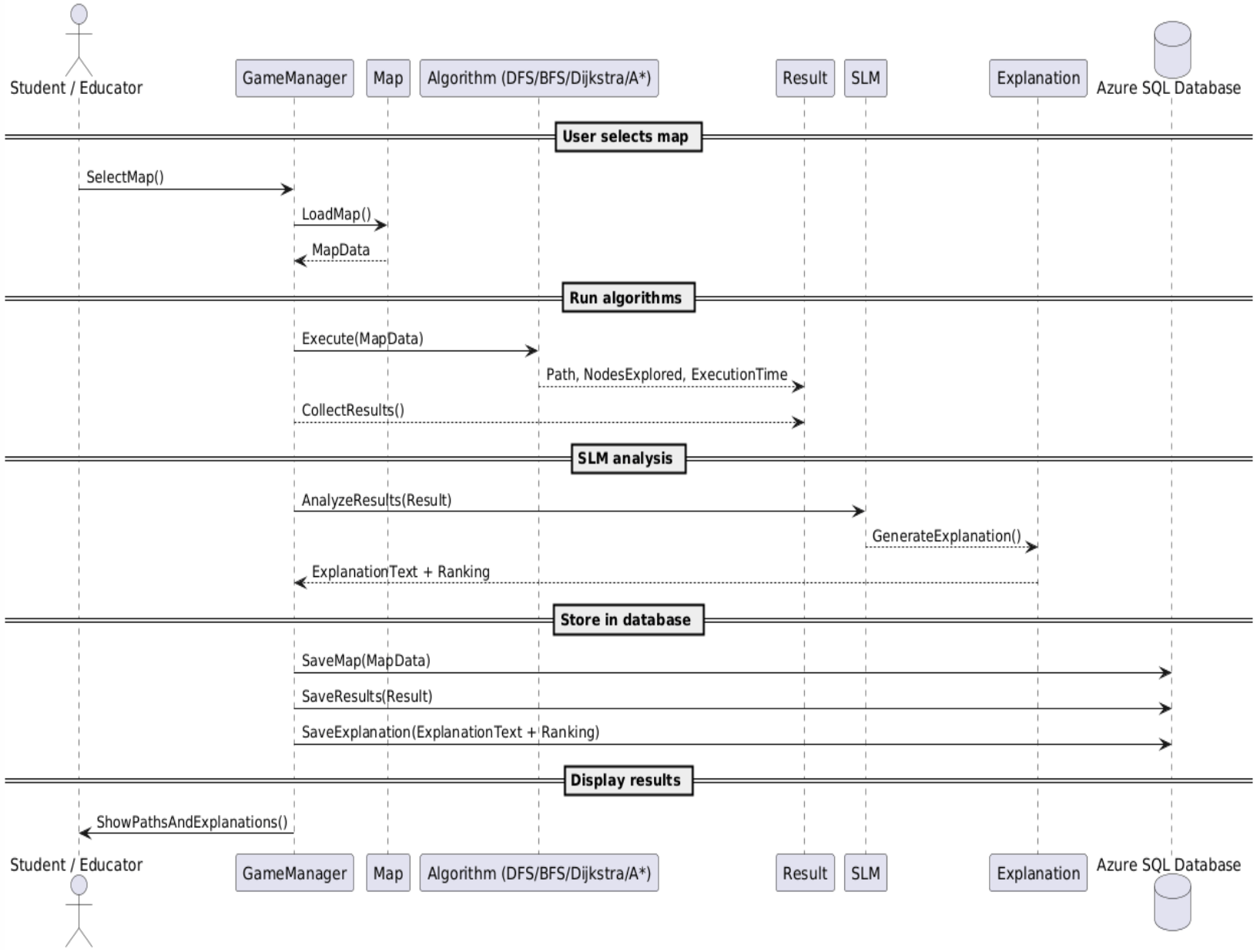
5.2 ERD diagrams



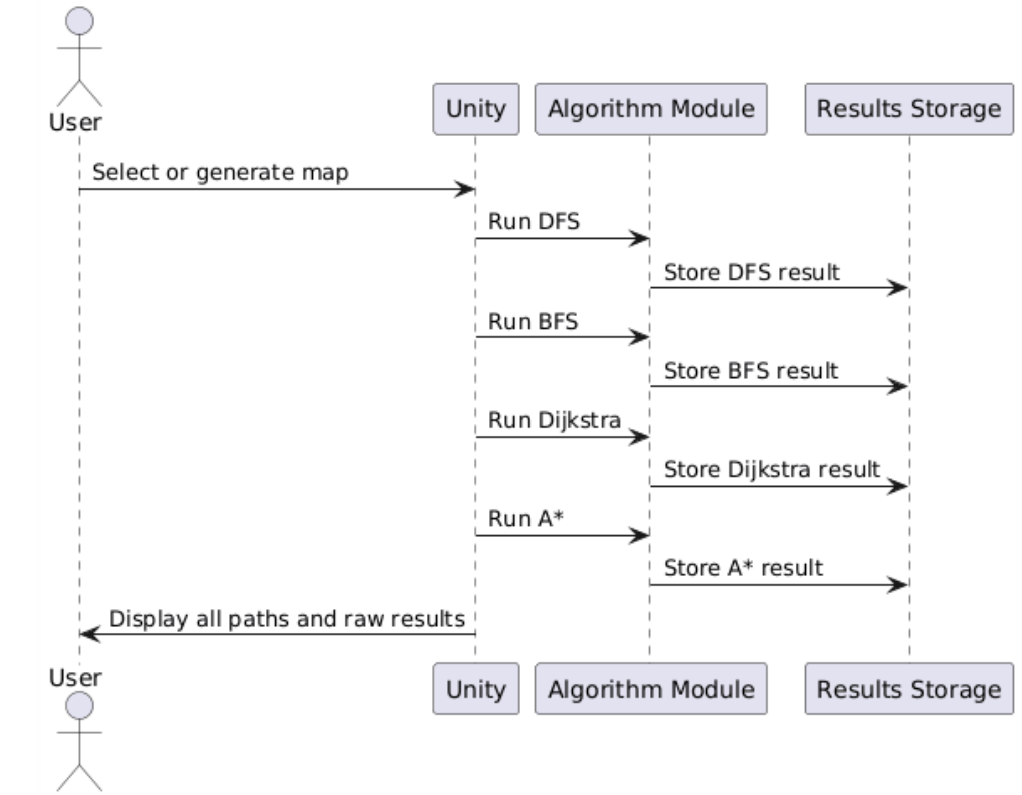
5.3 Use cases



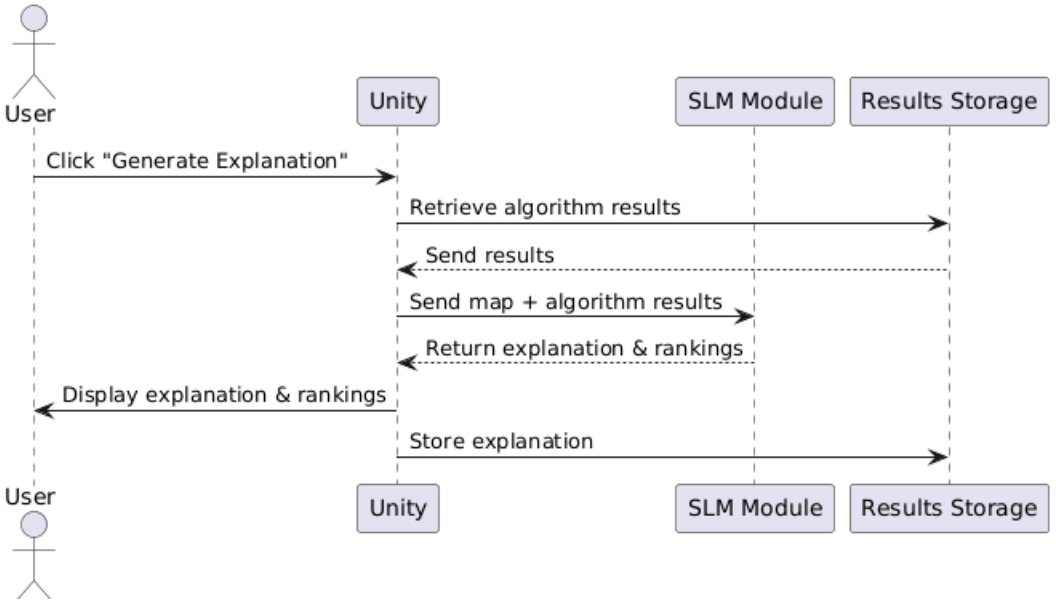
5.4 Sequence diagram



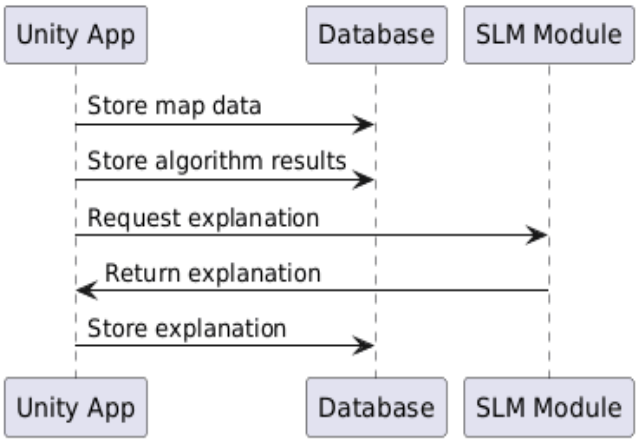
5.4.1 Run Pathfinding Algorithms



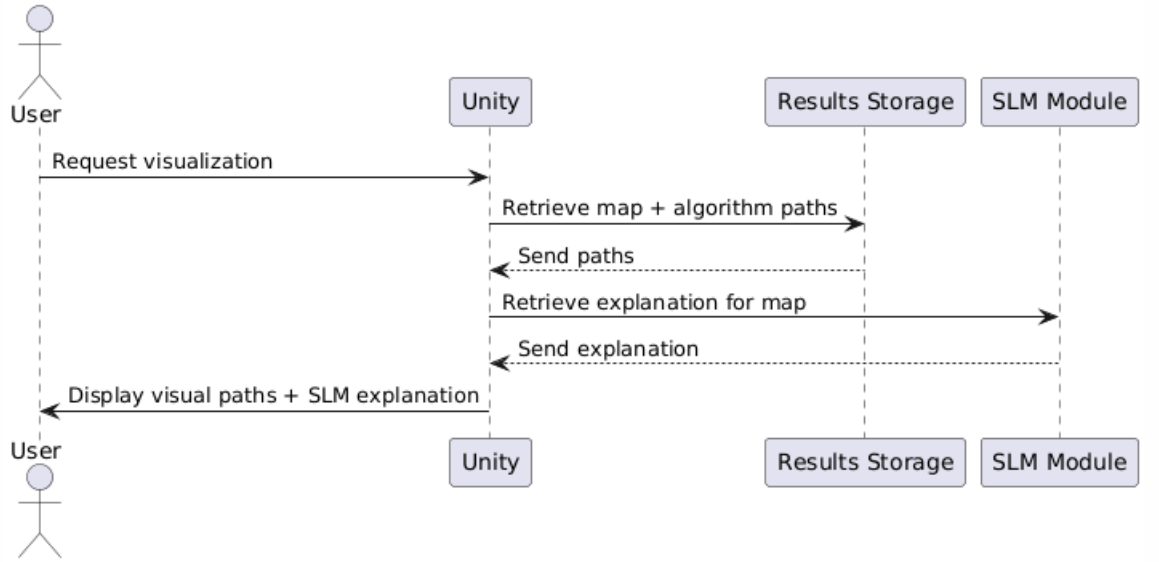
5.4.2 Generate AI Explanation



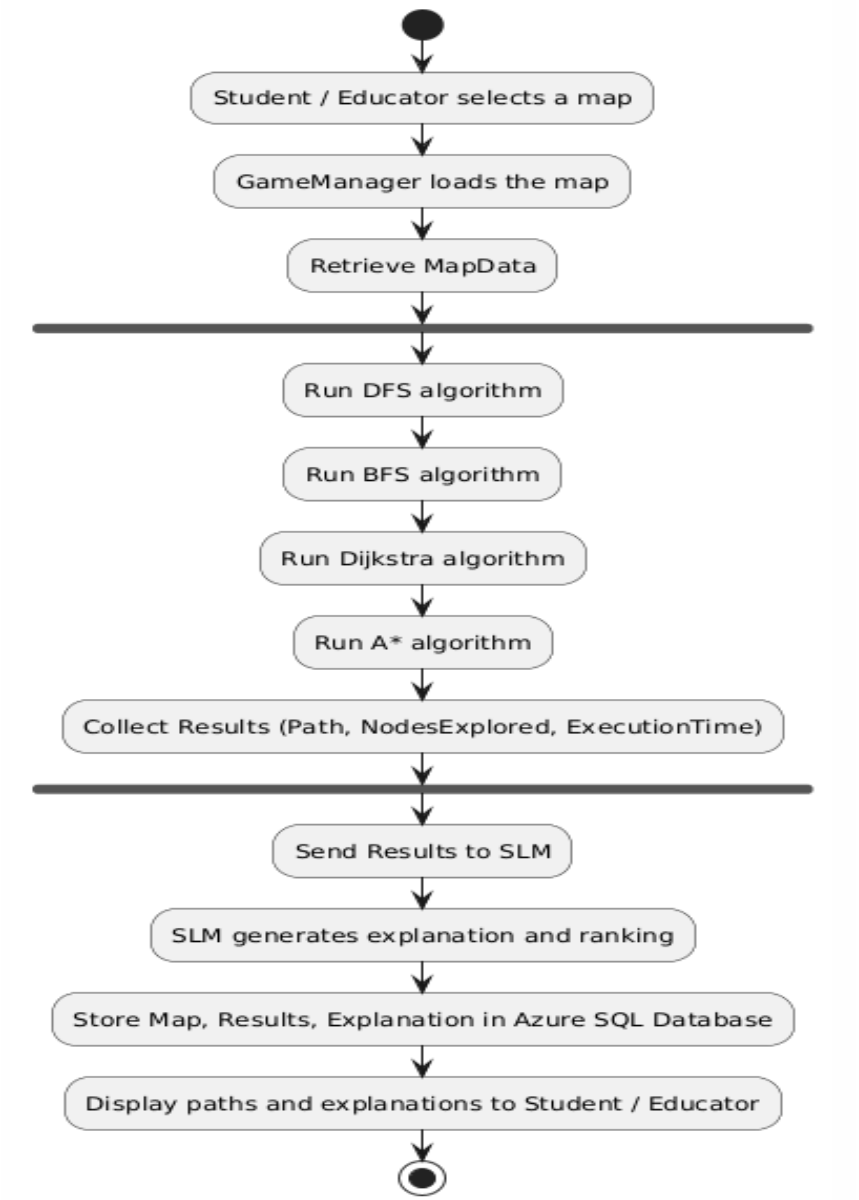
5.4.3 Store Results in Database



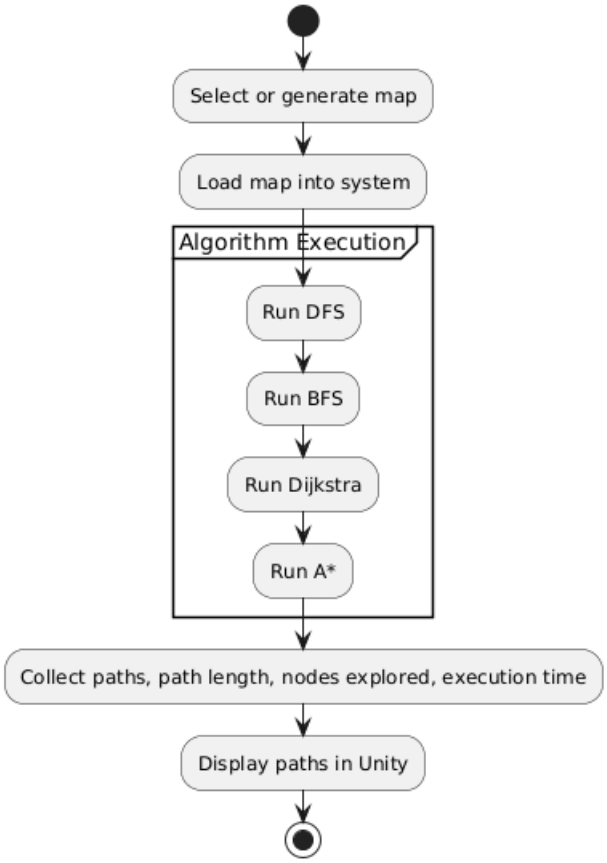
5.4.4 Visualize Results



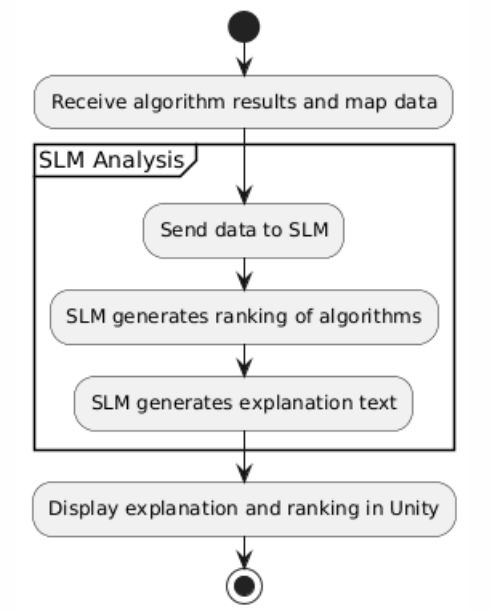
5.5 Activity diagrams



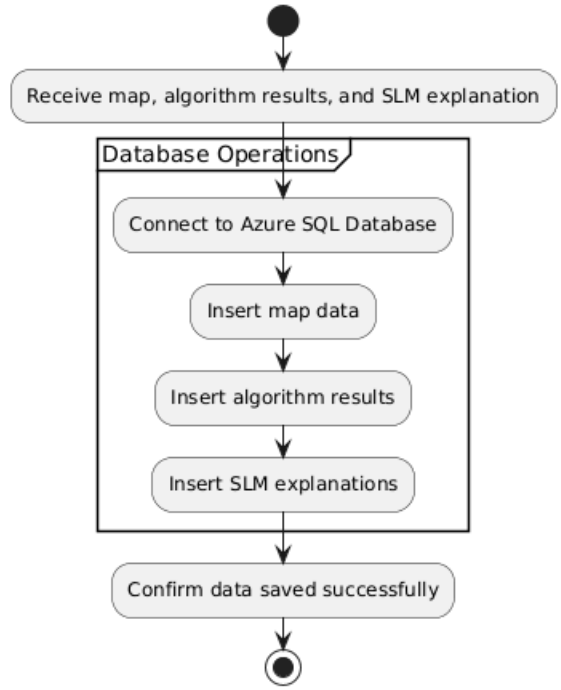
5.5.1 Run Pathfinding Algorithms on a Map



5.5.2 Generate AI Explanation



5.5.3 Store Results in Database



5.5.4 Visualize Results